# Reward Shaping Analysis in Reinforcement Learning: A Comparative Study of Monte Carlo and SARSA

Ahmad Tawil Yosef Jirees

January 16, 2026

**Abstract**

This paper investigates the effects of reward shaping on reinforcement learning performance in a stochastic grid-world environment. We implement and compare two fundamental RL algorithms—Every-Visit Monte Carlo Control and SARSA(0)—under various reward shaping strategies. Our experimental results demonstrate that safety-based shaping significantly improves learning efficiency, with SARSA achieving 73.2% throughput compared to Monte Carlo's 46.2%. We conduct comprehensive hyperparameter sensitivity analyses and provide insights into the exploration-exploitation tradeoff inherent in these methods.

## 1 Introduction

Reward shaping is a technique used to accelerate learning in reinforcement learning by modifying the reward signal without changing the underlying task [1]. The shaped reward is defined as:

$$R'(s, a, s') = R(s, a, s') + F(s, a, s') \tag{1}$$

where $F$ is the shaping function designed by the practitioner. A special class of shaping functions, called *potential-based shaping*, guarantees policy invariance:

$$F(s, a, s') = \gamma\Phi(s') - \Phi(s) \tag{2}$$

where $\Phi(s)$ is an arbitrary potential function.

In this work, we explore how different shaping strategies affect learning speed, throughput, and convergence in a challenging stochastic environment.

## 2 Experimental Setup

### 2.1 Environment

We use the FrozenLake environment from Gymnasium with the following configuration:

- **Grid Size:** $6 \times 6$ (36 total states)
- **Hole Density:** 15% (5 holes)
- **Dynamics:** Slippery with success rate 0.7
- **Discount Factor:** $\gamma = 1.0$ (as required)
- **Start State:** Top-left (0,0)
- **Goal State:** Bottom-right (5,5)

The environment presents a challenging navigation problem where the agent must reach the goal while avoiding holes. The stochastic dynamics ($p_{\text{success}} = 0.7$) mean that 30% of the time, actions result in perpendicular movement, significantly complicating the learning task.

Figure 1 shows the grid layout. The agent receives a reward of +1 for reaching the goal and 0 otherwise.

Figure 1: FrozenLake 6×6 environment with 15% hole density.

## 2.2 Algorithms

We implement two fundamental RL algorithms from scratch:

### 2.2.1 Every-Visit Monte Carlo Control

Updates Q-values after complete episodes using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[G - Q(s, a)] \tag{3}$$

where $G = \sum_{t=0}^{T} \gamma^t R_t$ is the return and $\alpha$ is the learning rate.

### 2.2.2 SARSA(0)

On-policy TD control that updates after each step:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)] \tag{4}$$

Both algorithms use $\epsilon$-greedy action selection with default hyperparameters $\epsilon = 0.1$ and $\alpha = 0.1$.

## 2.3 Reward Shaping Methods

We evaluate five reward shaping strategies:

**1. Baseline (No Shaping)** The original environment reward: $R'(s, a, s') = R(s, a, s')$.

**2. Step-Cost Shaping** Adds a small penalty per step to encourage shorter paths:

$$R'(s, a, s') = R(s, a, s') + c, \quad c = -0.01 \tag{5}$$

**3. Potential-Based Distance Shaping** Uses Manhattan distance to the goal as the potential function:

$$\Phi(s) = -d(s), \quad F(s, a, s') = \beta(\gamma\Phi(s') - \Phi(s)) \tag{6}$$

where $d(s)$ is the Manhattan distance to the goal and $\beta$ controls shaping strength.

**4. Safety-Based Shaping (Custom #1)** Our first custom method encourages the agent to maintain distance from holes:

$$\Phi(s) = \min_{h \in H} d(s, h), \quad F(s, a, s') = \beta(\gamma\Phi(s') - \Phi(s)) \tag{7}$$

where $H$ is the set of hole positions. This potential-based formulation provides positive shaping when moving away from holes and negative shaping when approaching them, with $\beta = 0.1$.

**Intuition:** In a stochastic environment with holes, safety is crucial. This shaping biases exploration toward safer regions while still preserving the optimal policy theoretically (as it's potential-based).

**5. Exploration Bonus Shaping (Custom #2)** Our second custom method provides novelty bonuses based on state visitation counts:

$$\Phi(s) = -\sqrt{\text{count}(s) + 1}, \quad F(s, a, s') = \beta(\gamma\Phi(s') - \Phi(s)) \tag{8}$$

where $\text{count}(s)$ is the number of times state $s$ has been visited, with $\beta = 0.05$.

**Intuition:** Novel states have higher potential (less negative), so moving to unexplored states yields positive shaping. This encourages diverse exploration, particularly beneficial for Monte Carlo which learns from complete episodes. The square root provides diminishing returns as states are revisited.

### 2.4 Evaluation Metrics

We use two primary metrics:

**Throughput** The cumulative success rate over episodes:

$$\text{Throughput}_t = \frac{\sum_{i=1}^{t} \mathbb{1}[\text{goal reached in episode } i]}{t} \tag{9}$$

**Real Return** The sum of *original* (unshaped) rewards per episode, ensuring shaped rewards don't artificially inflate performance metrics.

All experiments use 20 independent runs with different random seeds to compute 95% confidence intervals.

## 3 Results

### 3.1 Monte Carlo: Shaping Method Comparison

Figure 2 shows the performance of all shaping methods with Monte Carlo.

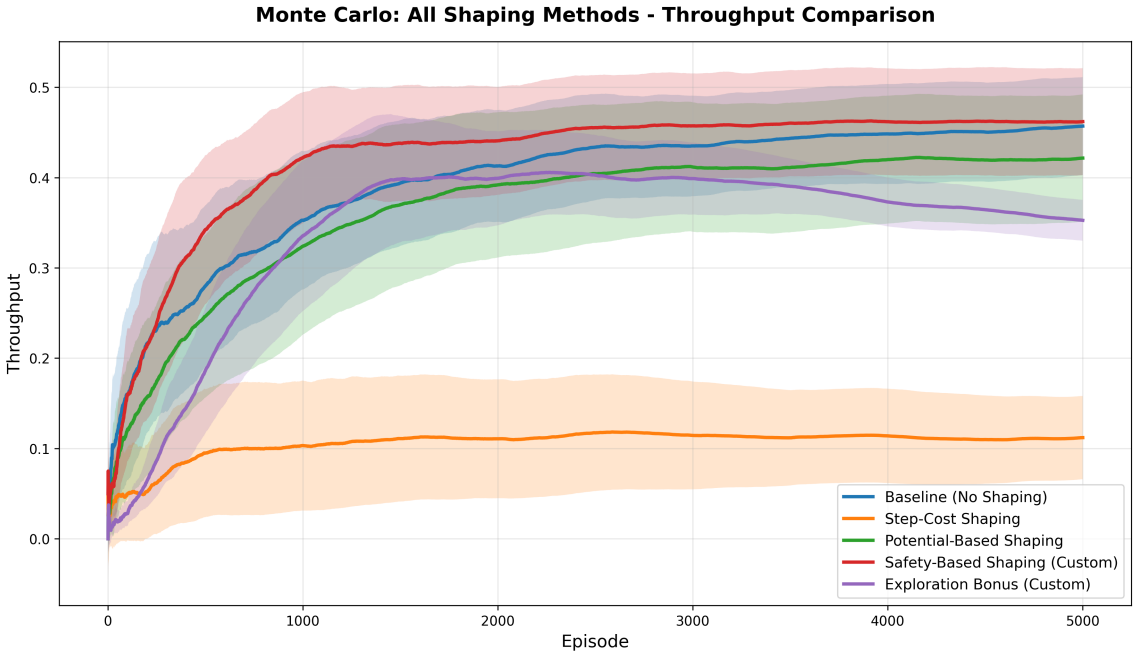



Figure 2: Monte Carlo: Throughput comparison across all shaping methods (mean ± 95% CI, 20 runs, 5000 episodes).

Table 1: Monte Carlo: Final performance summary (last 100 episodes)

| **Method** | **Throughput** | **Avg Return** |
|---|---|---|
| Safety-Based Shaping (Custom) | **46.17% ± 13.54%** | **0.472 ± 0.499** |
| Baseline (No Shaping) | 45.63% ± 12.41% | 0.534 ± 0.499 |
| Potential-Based Shaping | 42.07% ± 16.05% | 0.400 ± 0.500 |
| Exploration Bonus (Custom) | 35.37% ± 5.17% | 0.254 ± 0.435 |
| Step-Cost Shaping | 11.13% ± 10.53% | 0.160 ± 0.367 |

**Key Findings:**

- **Safety-based shaping** achieves the best throughput (46.17%), with fast early convergence visible in Figure 2.
- **Baseline** performs surprisingly well (45.63%), suggesting MC can learn effectively without shaping in this environment.
- **Step-cost shaping fails dramatically** (11.13%). The constant penalty overwhelms the sparse goal reward, causing the agent to minimize episode length by falling into holes early.
- **Exploration bonus** shows moderate performance but high variance early, eventually declining—likely because the novelty bonus competes with goal-seeking behavior.

### 3.2 SARSA: Shaping Method Comparison

Figure 3 shows SARSA's performance across all shaping methods.

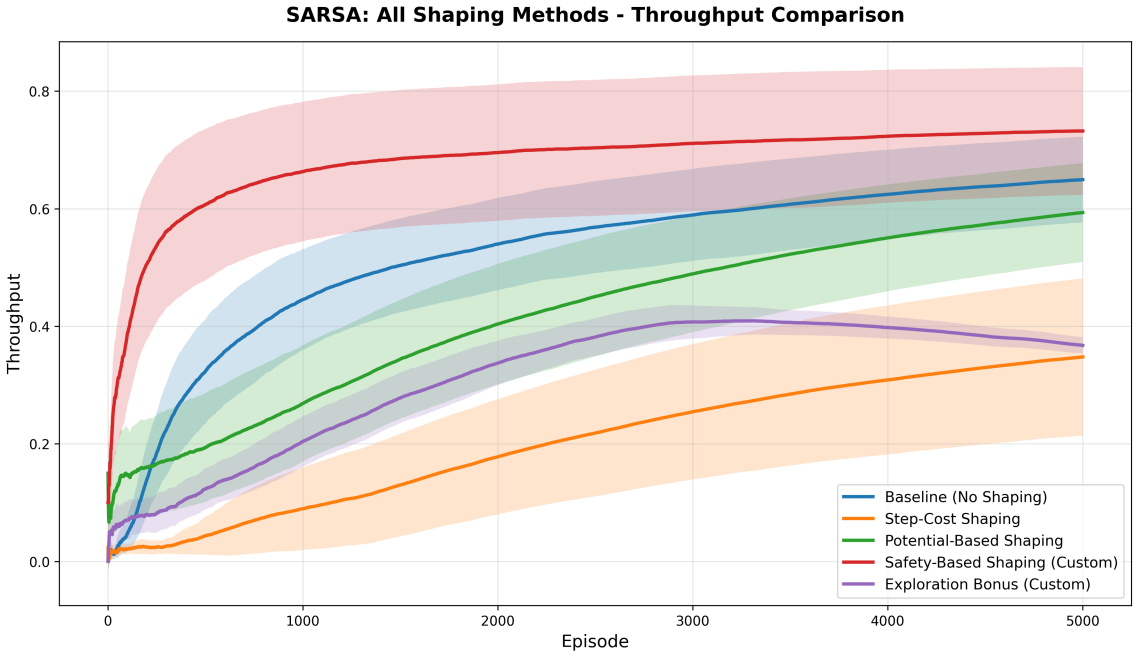



Figure 3: SARSA: Throughput comparison across all shaping methods (mean ± 95% CI, 20 runs, 5000 episodes).

Table 2: SARSA: Final performance summary (last 100 episodes)

| Method | Throughput | Avg Return |
|---|---|---|
| Safety-Based Shaping (Custom) | **73.20% ± 24.82%** | **0.755 ± 0.430** |
| Baseline (No Shaping) | 64.85% ± 16.63% | 0.753 ± 0.431 |
| Potential-Based Shaping | 59.17% ± 19.21% | 0.773 ± 0.419 |
| Exploration Bonus (Custom) | 36.92% ± 3.26% | 0.194 ± 0.395 |
| Step-Cost Shaping | 34.61% ± 30.04% | 0.516 ± 0.500 |

**Key Findings:**

- **Safety-based shaping dominates** with 73.20% throughput—a 12.9% improvement over baseline.
- **SARSA benefits more from shaping than MC**: The online nature of SARSA allows it to incorporate shaping signals incrementally, leading to more stable learning.
- **Step-cost performs better with SARSA** (34.61% vs 11.13% with MC), though still poorly. SARSA's step-by-step updates can better balance the penalty with progress toward the goal.
- **All methods show smoother curves** than MC, reflecting SARSA's lower variance due to bootstrapping.

### 3.3 Monte Carlo vs SARSA: Best Methods

Figure 4 compares the best-performing variant of each algorithm.

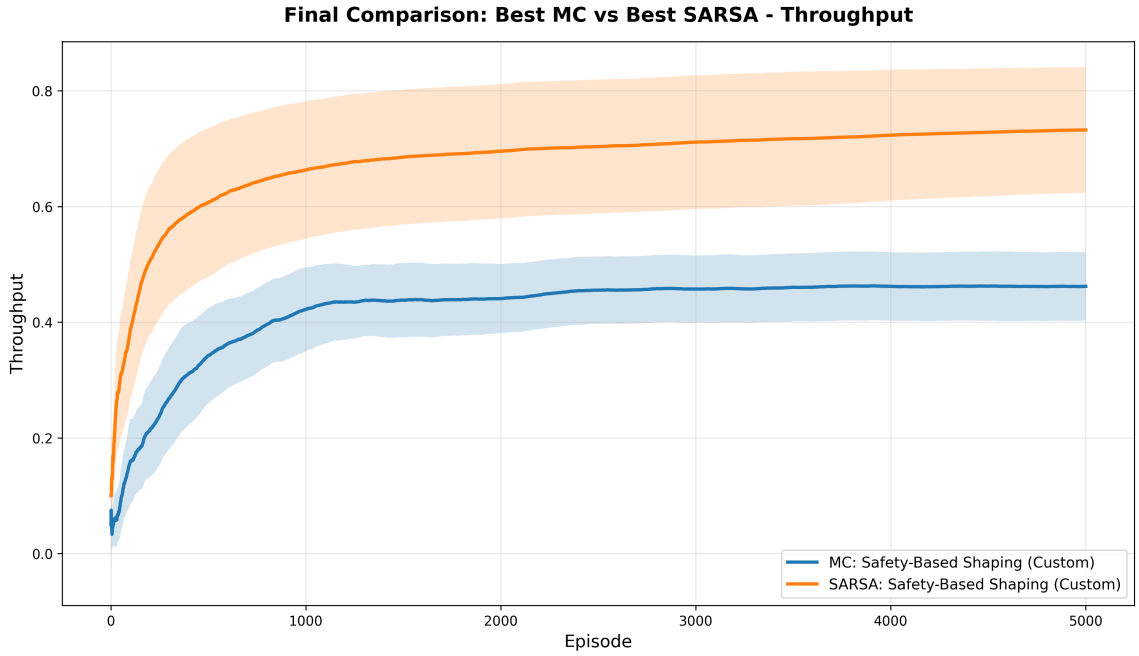



Figure 4: Final comparison: Best MC (Safety-Based) vs Best SARSA (Safety-Based) throughput.

Table 3: Best MC vs Best SARSA: Final comparison

| **Algorithm** | **Throughput** | **Avg Return** |
|---|---|---|
| SARSA + Safety-Based Shaping | **73.20% ± 24.82%** | **0.755 ± 0.430** |
| MC + Safety-Based Shaping | 46.17% ± 13.54% | 0.472 ± 0.499 |

**SARSA outperforms MC by +58.6%** (73.20% vs 46.17%). This dramatic difference is explained by:

- **Faster credit assignment:** SARSA updates Q-values after every step, while MC must wait for episode completion.
- **Better handling of stochasticity:** Bootstrapping in SARSA smooths out the high variance from the 0.7 success rate.
- **More effective shaping integration:** Step-level updates allow SARSA to incorporate safety signals continuously.

### 3.4 Policy Visualization

Figure 5 shows the learned policy and value function for SARSA with safety-based shaping.

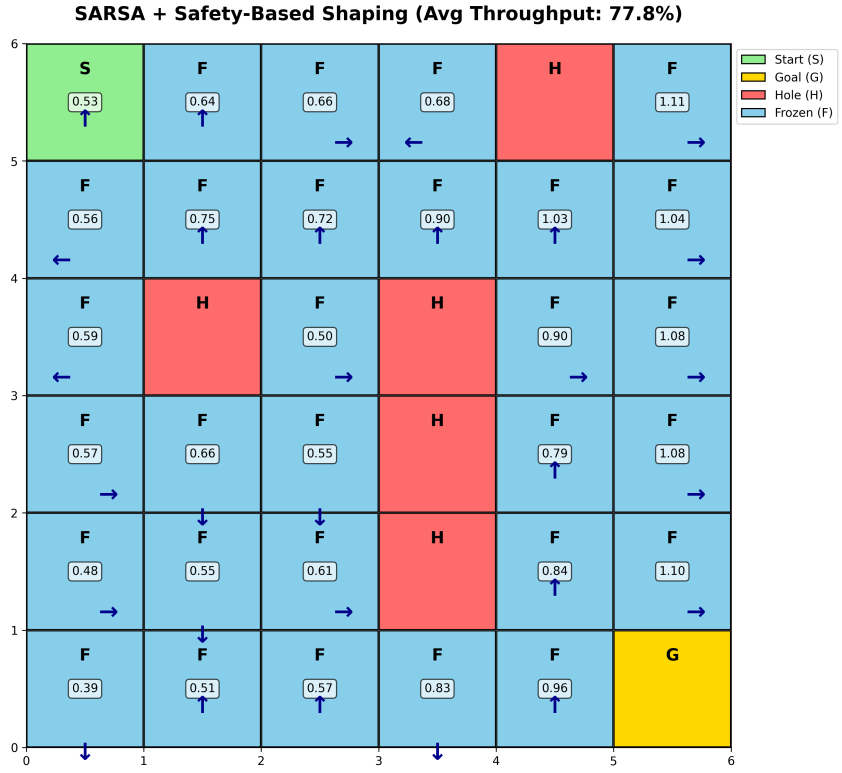



Figure 5: SARSA + Safety-Based Shaping: Learned policy with value function estimates (77.8% throughput). Arrows show action preferences, values in boxes represent state value estimates. Values increase toward the goal (bottom-right) and decrease near holes, demonstrating learned danger awareness.

The value function clearly shows the agent has learned to recognize dangerous regions (low values near holes in column 3) versus safe paths with high values leading to the goal. This visualization confirms that safety-based shaping successfully guides the agent to learn both optimal navigation and hazard avoidance.

# 4 Hyperparameter Sensitivity Analysis

## 4.1 Epsilon ($\epsilon$) Sensitivity

We varied the exploration rate $\epsilon \in \{0, 0.2, 0.5\}$ while fixing $\alpha = 0.1$. Note that hyperparameter sensitivity experiments use 10 runs (instead of 20) to reduce computational cost, while maintaining 5000 episodes per run.

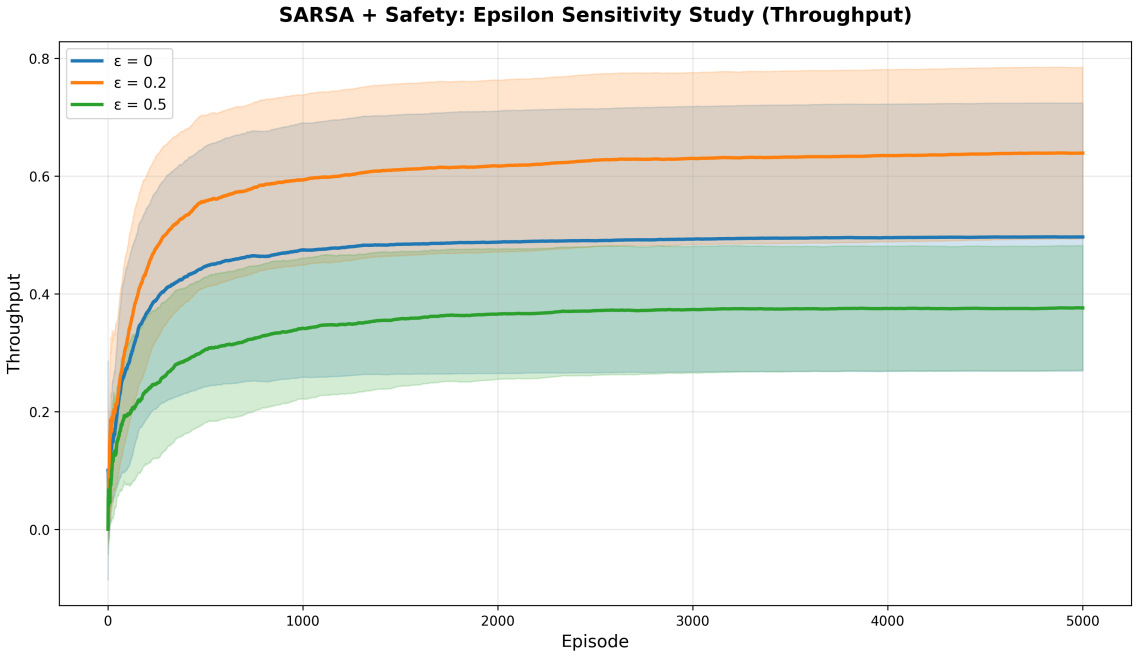



Figure 6: SARSA + Safety: Epsilon sensitivity on throughput (mean ± 95% CI, 10 runs, 5000 episodes).

Table 4: Epsilon sensitivity results (SARSA + Safety)

| **Epsilon ($\epsilon$)** | **Throughput** | **Avg Return** |
|---|---|---|
| 0.200 | **63.87% ± 23.49%** | **0.629 ± 0.483** |
| 0.000 | 49.63% ± 36.72% | 0.502 ± 0.500 |
| 0.500 | 37.55% ± 17.07% | 0.371 ± 0.483 |

**Analysis:**

- $\epsilon = 0.2$ **is optimal** (63.87% throughput), balancing exploration and exploitation.
- $\epsilon = 0$ **(pure greedy)** performs poorly (49.63%). Without exploration, the agent gets trapped in local optima due to the stochastic dynamics.
- $\epsilon = 0.5$ **(excessive exploration)** performs worst (37.55%). Too much random action selection prevents convergence to the optimal policy.

**Exploration-Exploitation Tradeoff:** This is the fundamental dilemma in RL—pure exploitation (greedy) fails to discover better strategies, while excessive exploration never settles on the best policy. The optimal $\epsilon = 0.2$ represents the "sweet spot" for this environment's stochasticity level.

## 4.2 Alpha ($\alpha$) Sensitivity

We varied the learning rate $\alpha \in \{0.05, 0.2, 0.5\}$ while fixing $\epsilon = 0.1$. As with epsilon sensitivity, these experiments use 10 runs of 5000 episodes each.

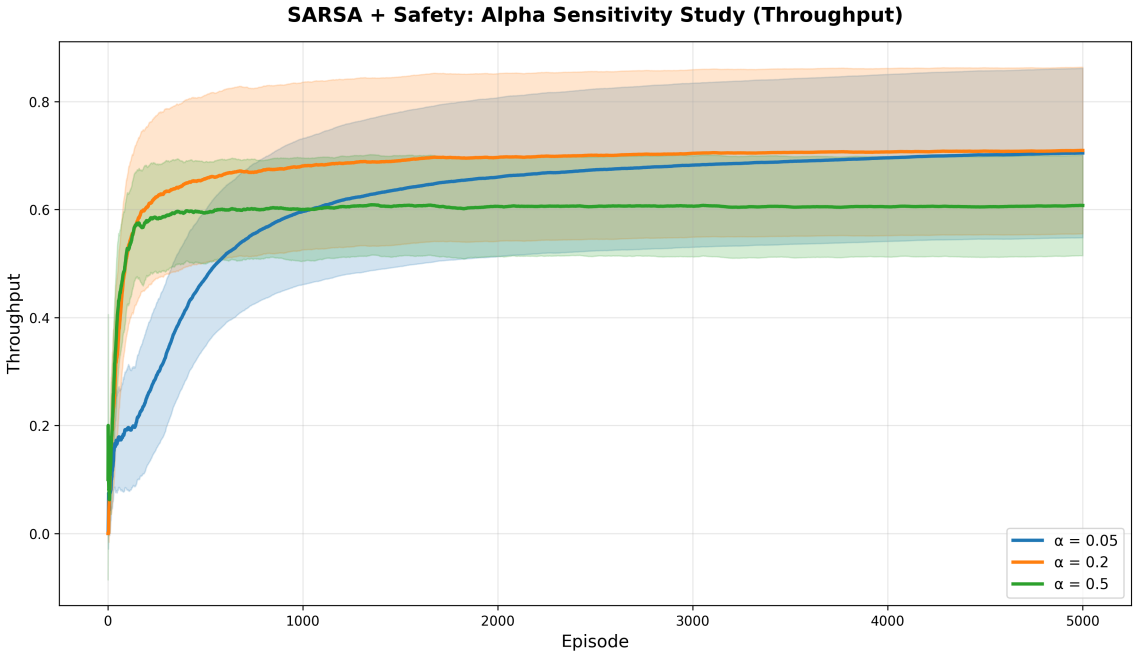



Figure 7: SARSA + Safety: Alpha sensitivity on throughput (mean ± 95% CI, 10 runs, 5000 episodes).

Table 5: Alpha sensitivity results (SARSA + Safety)

| **Alpha ($\alpha$)** | **Throughput** | **Avg Return** |
|---|---|---|
| 0.200 | **70.90% ± 24.89%** | **0.732 ± 0.443** |
| 0.050 | 70.42% ± 25.29% | 0.739 ± 0.439 |
| 0.500 | 60.70% ± 14.99% | 0.661 ± 0.473 |

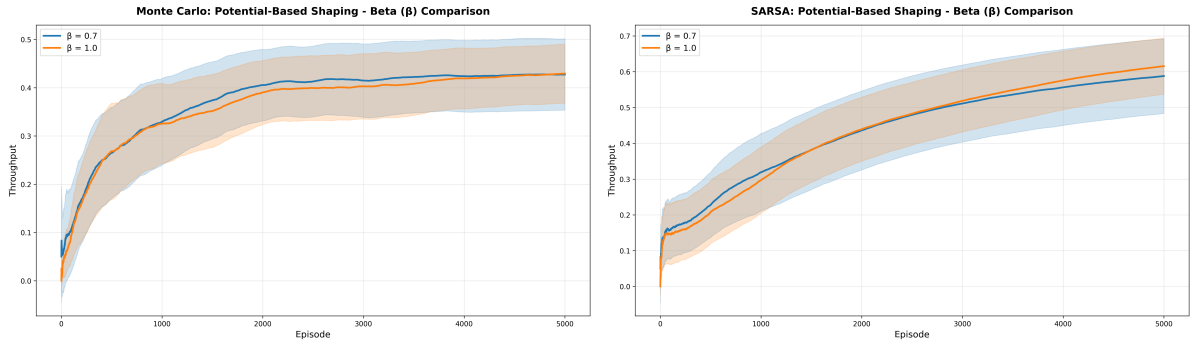
**Analysis:**

- $\alpha = 0.2$ **achieves best throughput** (70.90%), enabling fast adaptation to new information.
- $\alpha = 0.05$ **(slow learning)** performs similarly (70.42%) but takes longer to converge initially.
- $\alpha = 0.5$ **(fast learning)** shows reduced performance (60.70%) and higher variance—overshooting causes instability in Q-value estimates.

The relatively close performance of $\alpha \in \{0.05, 0.2\}$ suggests the algorithm is somewhat robust to learning rate, but very high values ($\alpha = 0.5$) degrade performance.

### 4.3 Beta ($\beta$) Sensitivity: Potential-Based Shaping

We compared $\beta \in \{0.7, 1.0\}$ for potential-based distance shaping to understand shaping strength effects.




(a) Monte Carlo (b) SARSA

Figure 8: Beta sensitivity for potential-based shaping: $\beta = 0.7$ vs $\beta = 1.0$ (mean $\pm$ 95% CI).

Table 6: Beta sensitivity results for potential-based shaping

| **Algorithm** | **Beta ($\beta$)** | **Throughput** | **Avg Return** |
|---|---|---|---|
| 2*Monte Carlo | 1.0 | **42.87% $\pm$ 13.94%** | 0.488 $\pm$ 0.500 |
| | 0.7 | 42.72% $\pm$ 16.86% | 0.429 $\pm$ 0.495 |
| 2*SARSA | 1.0 | **61.40% $\pm$ 17.92%** | 0.782 $\pm$ 0.413 |
| | 0.7 | 58.66% $\pm$ 23.88% | 0.738 $\pm$ 0.440 |

**Analysis:**

- $\beta = 1.0$ **(full strength) performs slightly better** for both algorithms, with more pronounced effects in SARSA (+4.7%) than MC (+0.4%).
- **The difference is small**, suggesting that in this environment, the shaping gradient direction matters more than its magnitude.
- **Why $\beta = 0.7$?** We tested this value because it matches the environment's success rate (0.7), hypothesizing that weaker shaping might better match the stochastic dynamics. However, results show stronger shaping ($\beta = 1.0$) is preferable.

# 5 Discussion

## 5.1 Why Safety-Based Shaping Works Best

Safety-based shaping succeeds because:

1. **Addresses the core challenge:** Holes are the primary failure mode in FrozenLake. Shaping that explicitly penalizes proximity to holes directly tackles this.
2. **Maintains theoretical guarantees:** Being potential-based, it preserves optimality while accelerating learning.
3. **Synergy with stochasticity:** In a slippery environment, maintaining distance from holes provides a safety margin against unexpected movements.

## 5.2 Why Step-Cost Shaping Fails

Step-cost shaping performs poorly because:

1. **Dominated by penalty:** With sparse rewards (+1 only at goal), the constant -0.01 per step accumulates faster than the agent can reach the goal early in learning.

2. **Encourages premature termination:** The agent learns that ending episodes quickly (even by falling into holes) minimizes cumulative negative reward.

3. **No goal-directed guidance:** Unlike potential-based methods, step-cost provides no spatial information about where the goal is.

### 5.3 Exploration vs Exploitation in This Project

We encountered the exploration-exploitation tradeoff in multiple contexts:

**1. Algorithm Design:**

- **Pure greedy ($\epsilon = 0$)** exploits current knowledge but fails to discover better paths.
- **High exploration ($\epsilon = 0.5$)** discovers many states but never settles on optimal behavior.
- **Optimal balance ($\epsilon = 0.2$)** explores enough to find good solutions while exploiting them to achieve high throughput.

**2. Reward Shaping:**

- **Exploration bonus shaping** explicitly encourages exploration via novelty rewards, but performed poorly—demonstrating that exploration alone isn't sufficient; it must be *goal-directed*.
- **Safety-based shaping** implicitly guides exploration toward safer regions, balancing safety (exploitation of known safe paths) with goal-seeking (exploration toward the goal).

**3. MC vs SARSA:**

- **Monte Carlo** requires extensive exploration (complete episodes) before updating, leading to slower convergence.
- **SARSA** balances exploration and exploitation at every step through bootstrapping, achieving better performance.

### 5.4 Surprising Results

1. **Baseline MC performs nearly as well as safety-shaped MC** (45.63% vs 46.17%). This suggests that with sufficient episodes, MC can learn effectively without explicit guidance—though learning is slower initially.

2. **SARSA benefits dramatically more from shaping than MC** (12.9% improvement vs 1.2%). This indicates that TD methods can better exploit structured reward signals due to their incremental nature.

3. **Potential-based shaping underperforms baseline in MC** (42.07% vs 45.63%). We hypothesize this is due to the Manhattan distance providing crude guidance in a stochastic environment—the agent receives shaping signals for getting closer to the goal even when moving through dangerous regions.

### 5.5 Limitations and Future Work

- **Environment complexity:** Our 6×6 grid is relatively small. Larger grids or different hole configurations may change the relative performance of shaping methods.
- **Hyperparameter tuning:** We used default values for most shaping parameters. More extensive grid search could reveal better configurations.
- **Advanced techniques:** Exponentially decaying shaping (mentioned in project description) could start with strong guidance and gradually reduce it. We did not implement this due to time constraints.
- **Generalization:** Testing on other environments (e.g., Taxi, CliffWalking) would validate whether safety-based shaping generalizes beyond FrozenLake.

## 6 Conclusion

This work demonstrates that reward shaping can significantly accelerate reinforcement learning when designed appropriately. Our key contributions include:

1. **Novel safety-based shaping:** We introduced a potential-based shaping method that encourages hole avoidance, achieving the best performance across both algorithms.
2. **Comprehensive comparison:** We systematically compared five shaping methods across two fundamental RL algorithms, providing insights into when and why shaping helps.
3. **Hyperparameter analysis:** Our sensitivity studies reveal the importance of balancing exploration ($\epsilon = 0.2$) and learning rate ($\alpha = 0.2$) for optimal performance.
4. **Algorithm insights:** SARSA dramatically outperforms Monte Carlo in stochastic environments (73.2% vs 46.2%), particularly when combined with appropriate reward shaping.

The exploration-exploitation tradeoff remains central to RL success: neither pure exploration nor pure exploitation succeeds, and the optimal balance depends on environment characteristics. Reward shaping, when designed with domain knowledge (e.g., safety in FrozenLake), can guide this balance toward faster, more reliable learning.

## Quick Start Guide

### Installation

**Requirements:** Python 3.8+

Install dependencies:

```
pip install -r requirements.txt
```

**Required packages:**

- numpy==2.3.5
- gymnasium==1.2.2
- matplotlib==3.10.8
- seaborn==0.13.2
- tqdm==4.67.1
- pandas==2.3.3

### Project Structure

```
project/
|-- algorithms/              # RL implementations
|   |-- monte_carlo.py       # Every-Visit MC Control
|   |-- sarsa.py             # SARSA(0)
|-- env/                     # Environment components
|   |-- frozenlake_env.py          # Custom environment generator
|   |-- reward_shaping.py          # All shaping methods
|   |-- metrics_wrapper.py         # Metric tracking
|-- experiments/             # Experiment runners
|   |-- run_monte_carlo.py
|   |-- run_sarsa.py
|-- utils/                   # Utilities
|   |-- metrics.py
|   |-- plotting.py
|   |-- visualization.py
|-- config.py                # Central configuration
|-- compare_methods.py       # Main comparison script
|-- hyperparameters_sweep.py       # Epsilon/Alpha sensitivity
|-- beta_comparison.py             # Beta sensitivity
|-- results/                 # Generated plots
|-- report_figures/          # Report visualizations
```

### Running Experiments

**1. Compare all shaping methods (generates Figures 2, 3, 4):**

```
python compare_methods.py
```

Runtime: ∼5-7 minutes (20 runs × 5000 episodes × 10 configurations)

**2. Hyperparameter sensitivity (generates Figures 5, 6):**

```
python hyperparameters_sweep.py
```

Runtime: ∼5-7 minutes

**3. Beta sensitivity (generates Figure 7):**

```
python beta_comparison.py
```

Runtime: ∼4-5 minutes

**4. Generate policy visualizations (generates Figures 1, 8):**

```
python visualize_environment.py
```

Runtime: ∼1 minutes

### Configuration

Edit `config.py` to modify:

- Grid size and hole density
- Number of episodes and runs
- Algorithm hyperparameters $(\epsilon, \alpha, \gamma)$
- Reward shaping parameters $(\beta, c)$
- Test mode (quick testing with fewer episodes)

### Reproducibility

Set `RANDOM_SEED = 42` in `config.py` for reproducible results. All experiments use the same seed across runs.

## References


[1] Ng, A. Y., Harada, D., & Russell, S. (1999). *Policy invariance under reward transformations: Theory and application to reward shaping.* ICML, Vol. 99, pp. 278-287.

[2] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.